

[← Go Back](#)

Hardware

Nicla Vision

Tutorials

[Blob Detection with OpenMV](#)[Smart Elevator Monitoring System with the Portenta Proto Kit](#)[Getting Started with Nicla Vision](#)[Image Classification with Edge Impulse®](#)[Debugging with Lauterbach TRACE32 GDB Front-End Debugger](#)[Video Live Streaming with OpenMV and Nicla Vision](#)[Reading Audio Samples With the Onboard Microphone](#)[Accessing IMU Data on Nicla Vision](#)[Proximity Detection with Arduino Nicla Vision](#)[Voice Commands With The Arduino Speech Recognition Engine](#)[Nicla Vision User Manual](#)[Home](#) / [Hardware](#) / [Nicla Vision](#) /

Smart Elevator Monitoring System with the Portenta Proto Kit

Smart Elevator Monitoring System with the Portenta Proto Kit

This application note describes how to implement a smart elevator monitoring system to measure air quality and occupancy.

Author · Christopher Méndez · Last revision · 01/07/2025

ON THIS PAGE

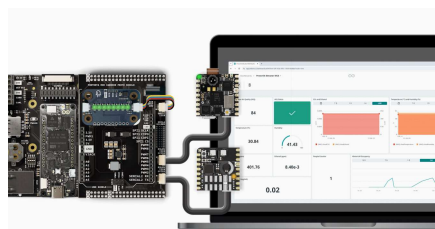
Introduction

[Goals](#)[Hardware and Software Requirements](#) +[Smart Elevator Monitoring System Setup](#) +[Smart Elevator Monitoring System Overview](#) +[Conclusions](#)[Next Steps](#)

Introduction

Air pollution is an often overlooked threat that exists even in the places we feel safest, from our homes to our workplaces. Pollutants like CO₂ and volatile organic compounds (TVOC) silently threaten our health, representing a determining factor for our quality of life.

This application note describes building an air quality monitor and occupancy tracking system using machine vision for indoor environments designed for use in an elevator.



The monitoring system will measure

[Help](#)

estimated CO₂ and total volatile organic compounds (TVOC) while counting the people inside the elevator. All this information will be forwarded to the Arduino Cloud for further visualization and ongoing monitoring. The system will provide real-time visual feedback of the condition inside the elevator to warn people of harmful air quality.

Goals

The project showcased in this application note has the following objectives:

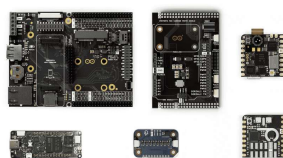
- Monitor the air quality and environmental conditions within the elevator.

- Count people inside the elevator using Machine Vision.

- Provide real-time visual feedback on the elevator conditions using Modulino® Pixels.

- Share all collected data to the Arduino Cloud for remote monitoring.

Hardware and Software Requirements



Hardware needed for the project

This project is meant to be developed with the **Portenta Proto Kit** that includes the following products among others:

[Nicla Sense Env](#) (x1)

[Nicla Vision](#) (x1)

[Portenta Mid Carrier](#) (x1)

[Portenta H7](#) (x1)

Modulino Pixels (x1)

Portenta Mid Carrier Proto Shield (x1)

[USB-C® cable](#) (x1)

Wi-Fi® W.FL antenna (x1)

ESLOV cable (x2)

Qwiic cable (x1)

Software Requirements

[Arduino IDE 2.0+](#) or [Arduino Web Editor](#)

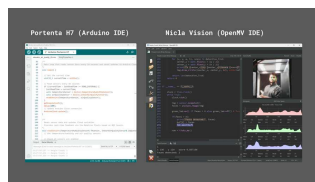
[OpenMV IDE](#)

[Arduino_NiclaSenseEnv library](#)

[Arduino Portenta Boards core](#)
(required to work with the Portenta H7 board)

[Arduino Cloud](#). If you do not have an account, create one for free [here](#).

Complete Project Sketch

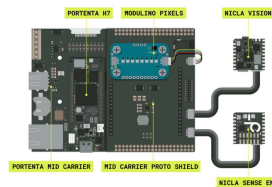


Project sketches

The complete project sketches can be downloaded [here](#):

Smart Elevator Monitoring System Setup

The electrical connections for the project are outlined in the diagram below:



System wiring overview

This diagram shows how the components are connected.

The Portenta H7 is attached to the Portenta Mid Carrier using the High-Density connectors.

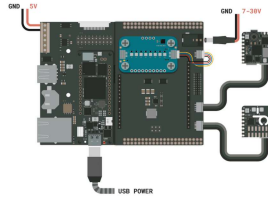
The Mid Carrier Proto Shield is stacked onto the Portenta Mid Carrier via breakout connectors.

The Modulino Pixels is wired to the Portenta Mid Carrier using the Qwiic cable and secured with the included screws and nuts.

The Nicla Vision and the Nicla Sense Env are connected to the Mid Carrier Proto Shield using ESLOV cables.

Powering Options

The project can be powered using the terminal block on the Portenta Mid Carrier, the USB-C connector on the Portenta H7 or the barrel jack on the



Powering options

Smart Elevator Monitoring System Overview

The elevator monitoring system integrates sensor data collection, machine vision, live user feedback and cloud connectivity via the Portenta H7's Wi-Fi® connection.

The **Portenta H7** is responsible for the Nicla Sense Env sensor readings, receiving occupant counts from the Nicla Vision through I2C, showing air quality status on the Modulino Pixels and communicating with the Cloud.

The **Nicla Vision** is the project's second system-on-module (SoM) dedicated exclusively to detecting faces in the elevator, counting them and reporting the count to the Portenta H7.

The **Nicla Sense Env** is a sensor shield that must be controlled by a host, in this case by the Portenta H7, and houses several environment sensors.

Indoor Air Quality Sensor

A key feature of this project is monitoring environmental variables, which is made possible by the

ZMOD4410 indoor air quality sensor featured within the **Nicla Sense Env.**

The **ZMOD4410** is a gas sensor module designed to detect total volatile organic compounds (TVOC), estimate CO₂ and monitor indoor air quality (IAQ) with ease of implementation.

The sensor can provide air quality data in three ways:

Renesas IAQ Rating: 0 - 5
(being 0 the cleaner and healthier air)

Interpreted Air Quality:
Qualitative ratings, as "Very Good", "Good", "Medium", "Poor" and "Bad"

Relative Air Quality: 0 - 500
(being 0 the cleaner and healthier air)

For this project, we will use the **Relative Air Quality** scale, as it offers an accurate and sensitive range to measure variations in air quality. Refer to the table below for reference:

AQI Level		Health Recommendation
Good	0-50	All people to unrestricted and enjoy their usual activities.
Moderate	51-100	People with respiratory issues should limit prolonged outdoor activities.
Unhealthy for Sensitive Groups	101-150	People with asthma and respiratory issues should limit prolonged outdoor activities.
Unhealthy	151-200	Increased likelihood of adverse effects and symptoms for most and very sensitive groups.
Very Unhealthy	201-300	Health risks significantly increase. Sensitive groups should avoid outdoor activities.
Hazardous	301-500	General public at high risk of respiratory illness. Sensitive groups should avoid outdoor activities. Avoid road travel unless critical.

Relative Air Quality table

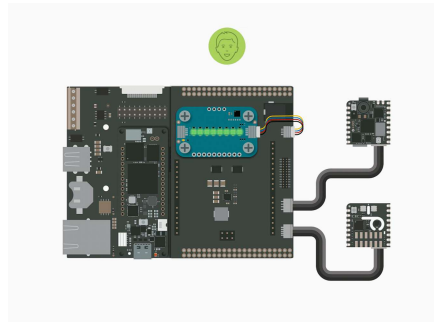
To complement the environment monitoring, we will also use the **HS4001** temperature and humidity sensor of the Nicla Sense Env.

Visual Air Quality Feedback

The system features real-time visual feedback to warn the occupants of the elevator so they know the current

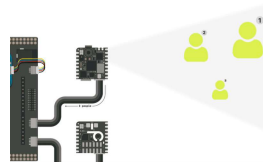
air quality status at any time and take the right preventive measures.

Using the Modulino Pixels, the system will display the colors corresponding to the Air Quality Index levels shown in the table above.



Machine Vision for People Counting

The Nicla Vision will use a built-in FOMO (Faster Objects, More Objects) model for face detection. With a simple Python script, it can report the face count within a single frame covering the entire elevator via I2C.



Nicla Vision people counting

Portenta H7 Code

Let's go through some important code sections to make this application fully operational, starting with the required libraries:

```
Arduino_NiclaSenseEnv.h
```

provides support for collecting data from the Nicla Sense Env

`Wire.h` enables I2C communication with the Nicla Sense Env and Nicla Vision boards.

`modulino.h` provides support for controlling the Modulino Pixels.

There is a header included in the project code for the Arduino Cloud configuration:

`thingProperties.h` includes the Wi-Fi® credentials and Arduino Cloud configuration.

This header includes two other libraries necessary for cloud communication, which are:

`ArduinoIoTCloud.h` enables Arduino Cloud integration. Search for *ArduinoIoTCloud* in the Library Manager to install it.

`Arduino_ConnectionHandler.h`

manages the board's internet connectivity. Search for *Arduino_ConnectionHandler* in the Library Manager to install it.

```
1 // Include necessary libr
2 #include "Arduino_NiclaSe
3
4 // Automatically generate
5 #include "thingProperties
6
7 // Include support for th
8 #include <Modulino.h>
9
10 // Include Wire library f
11 #include <Wire.h>
12
13 // Set time interval (in
14 static const uint32_t REA
15 uint32_t lastReadTime = 0
16
17 // Global Parameters
18 byte people = 0; // peop
19
20 // Sensor object for Nicl
21 NiclaSenseEnv device;
22
23 // Modulino Pixels object
24 ModulinoPixels leds;
25
26 // The Nicla Vision I2C a
27 #define I2C_ADDR 0x35
```

In the `setup()` function, a variety of peripherals are initialized, including:

- Serial communication
- I2C communication
- Arduino Cloud properties
- Nicla Sense Env communication and sensor objects
- Modulino Pixels

```
1 void setup() {
2   // Initialize serial c
3   Serial.begin(9600);
4
5   // Short delay to wait
6   delay(1500);
7
8   Wire.begin();
9
10  // Initialize Cloud pr
11  initProperties();
12  ArduinoCloud.begin(Arc
13
14  // Debug information
15  setDebugMessageLevel(2
16  ArduinoCloud.printDebu
17
18  // Initialize Nicla Se
19  if (device.begin()) {
20    Serial.println("- Ni
21
22    auto temperatureSens
23    auto airQualitySensc
24
25    // Set the sensor mc
26    airQualitySensor.set
27
28    // The ZMOD4410 can
```

In the `loop()` function, the Nicla Sense Env sensors are read every 10 seconds, the people count is requested to the Nicla Vision and the Cloud connection is updated.

```
1 void loop() {  
2  
3   // Get the current time  
4   uint32_t currentTime =  
5  
6   // Read sensors every 1  
7   if (currentTime - lastR  
8       lastReadTime = curren  
9       auto temperatureSens  
10      auto airQualitySensor  
11      readSensors(temperatu  
12  }  
13  getPeopleCount(); // re  
14  delay(100);  
15  // Update Arduino Cloud  
16  ArduinoCloud.update();  
17 }
```

One of the main functions in the `loop()` is `readSensors()`, which uses the Nicla Sense Env API to read the following variables:

- Temperature (°C)
- Relative Humidity (%)
- CO₂ (ppm)
- Ethanol (ppm)
- TVOC (mg/m3)
- Relative Air Quality (IAQ)

We also control the Modulino Pixels color based on the IAQ and update the cloud variables.

The `getPeopleCount()` function creates an I2C request asking for the count of people detected by the Nicla Vision.



You can download the complete example code for the Portenta H7 [here](#)

Nicla Vision Code

The people counting feature of the

running a FOMO face detection model.

For this feature, we are using the [OpenMV IDE](#) for running MicroPython sketches that you can download from [here](#).

Here's a brief overview of how the code works, starting with the main loop of the script:

```
1  if __name__ == "__main__":
2
3      clock = time.clock()
4      while True:
5          clock.tick()
6
7          img = sensor.snapshot()
8          faces = analyze_image(img)
9
10         green_led.on() if faces > 0 else green_led.off()
11
12         if faces > 0:
13             i2c.init(I2C_ADDRESS)
14             print("Faces detected: %d" % faces)
15             buf[0] = faces
16             i2c.send(buf)
17         else:
18             i2c.deinit()
19
20         now = ticks_ms()
```

In simple words, we are on an infinite loop taking pictures with the `sensor.snapshot()` function, which is then used as inputs for the face detection model using the `analyze_image()` function.

If faces are detected, the onboard green LED will light up and the count will be sent via I2C to the Portenta H7.

During the face detection process, several auxiliary functions filter unwanted results including false

positives. A brief explanation of these functions is listed below:

`calculate_distance()`: it returns the distance between two rectangles bounding a possible face to avoid duplicates.

`merge_rectangles()` and `merge_close_rectangles()`: Merges two or more close bounding rectangles into a single one if they are in proximity.

`fomo_post_process()`: it returns the list of bounding boxes to be analyzed by the previously explained functions.

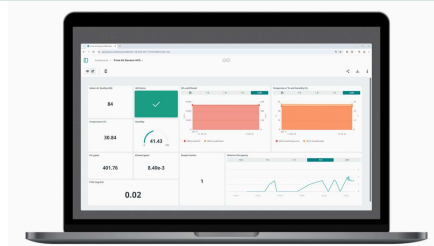
 You can download the complete example code for the Nicla Vision [here](#)

If you want to test the Nicla Vision example code **standalone**, you must comment out the `i2c.send(buf)` line in the sketch, this is because it will show an error if it can't find the Portenta H7 on the I2C bus.

 For the Nicla Vision to run the sketch once is powered up you must put the code inside the `main.py` file in the file system.

Arduino Cloud Dashboard

By leveraging the Arduino Cloud, we can seamlessly integrate a simple yet powerful dashboard to monitor and visualize the system status in real-time:



Within the Arduino Cloud's dashboard, the system variables can be monitored with the following widgets:

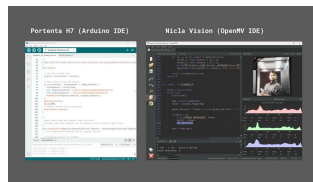
System variable gauge showing relative humidity.

Numeric state widgets to show IAQ, temperature, CO₂, ethanol and TVOC readings.

Advanced charts showing historical CO₂, ethanol, temperature and humidity records.

A people occupancy chart displaying both historical data and the current count.

Complete Project Sketch



Project sketches

The complete project sketches can be downloaded here:

[Portenta H7 code](#)

[Nicla Vision code](#)

Conclusions

In this application note, we have learned how to implement an indoor air quality and occupancy monitoring

conventional elevator into a smart, monitored space.

This application shows how Arduino's ecosystem simplifies the workflow for developing smart solutions to address real industrial needs. The Arduino PRO product line is a perfect fit for developing robust and reliable projects for the industry. We covered on-site sensor data sampling, machine vision, and real-time Cloud monitoring.

Next Steps

Now that you know how to develop an indoor air quality and people counting system with the Portenta Proto Kit, it is time to continue exploring all the capabilities of the Arduino Pro environment. Integrate it with your professional setup and enhance it with powerful solutions.

You can take this solution even further by:

Creating your own FOMO model using Edge Impulse to detect people particularly, rather than just faces.

Adding a Modulino Buzzer to provide an alert if the elevator occupancy exceeds the limit or if air quality deteriorates.

Integrating the Arduino 4G Module for deployments without Wi-Fi or Ethernet connectivity.

Suggest changes

The content on [docs.arduino.cc](#)

Need support?

[Help Center](#)
[Ask the Arduino Forum](#)

License

The Arduino documentation is licensed under the

through a
public
[GitHub](#)
repository.
If you see
anything
wrong, you
can edit this
page [here](#).

[Share Alike 4.0](#)
license.

Was this article helpful?

